

Signal Processing

Project 1

Submitted by: Miriam Chavay

Lecturer: Prof. Rafi Haddad

Submission date: 17.5.2026

Part 1

A.1 Signal Splitting and Graphical Display

In this section we split the recorded signal into two main parts according to time: the first segment, between 0 and 9 seconds, was defined as noise (baseline). The second segment, between 12 and 30 seconds, was defined as the stimulus signal (stimulus), which contains both the response to the stimulus and noise.

The split was performed based on the sampling rate (50 Hz), i.e., every second contains 50 samples, so times can be converted to indices in the data array. In addition, we built a time axis for each segment using a function that converts indices to seconds.

Presentation of the three graphs:

1. Noise graph (0–9 seconds) – shows the behavior of the noise alone.

Code snippet and graph display:

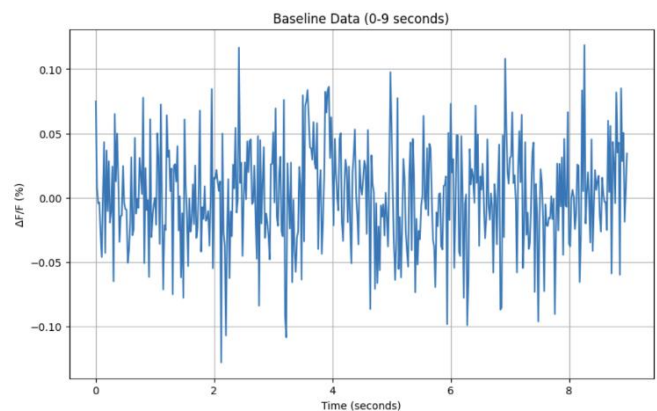
```
# sampling rate
HZ = 50

#first 9 seconds as baseline (noise)
baseline_end_time = 9
baseline_df = df.iloc[0 : baseline_end_time * HZ]

#signal starting from 12 seconds (stimulus part)
stim_start_time = 12
stim_df = df.iloc[stim_start_time * HZ : ]

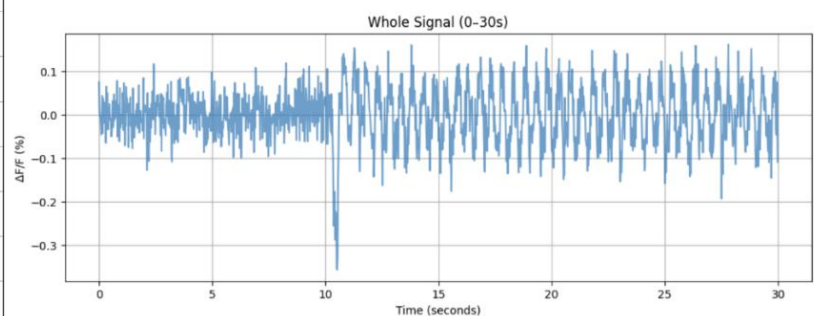
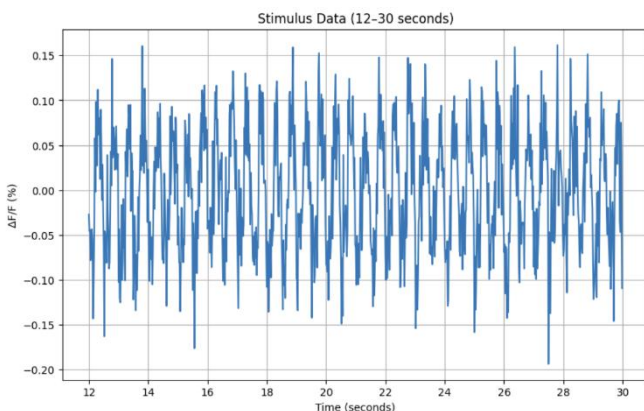
#time axis for plotting
def get_time_axis(data_frame, sampling_rate, start_offset=0):
    return (np.arange(len(data_frame)) / sampling_rate) + start_offset

#signals as numpy arrays
stim_signal = stim_df['ΔF/F (%)'].values # signal + noise
noise_signal = baseline_df['ΔF/F (%)'].values # noise only
```



2. Stimulus graph (12–30 seconds) – shows the signal while the stimulus was applied.
3. Full signal graph (0–30 seconds) – shows the entire signal continuously and allows a comprehensive view of the difference between the noise region and the stimulus region.

Graph display:



B.1 Resampling and the Effect of Sampling Rate on Signal Quality

The original signal was sampled at a rate of 50 Hz. To resample it at frequencies of 25 Hz, 10 Hz and 3.33 Hz, we calculated the step size according to the ratio between the original sampling rate and the target rate: for 25 Hz, take every 2nd point ($50/25 = 2$); for 10 Hz, every 5th point ($50/10 = 5$); and for ≈ 3.33 Hz, every 15th point ($50/15 \approx 3.33$). After resampling, we built a new time axis for each signal according to the actual sampling rate, in order to preserve a correct representation of time.

From the results it can be seen that for frequencies of 25 Hz and 10 Hz the signal retains its general shape, i.e., most of the information is preserved.

Code snippet and graph display:

1.B Manual Signal Resampling

```
#target sampling frequencies
target_frequencies = [25, 10, 3.33]

# use only the stimulus part
stimulus_part = stim_signal

for f_new in target_frequencies:

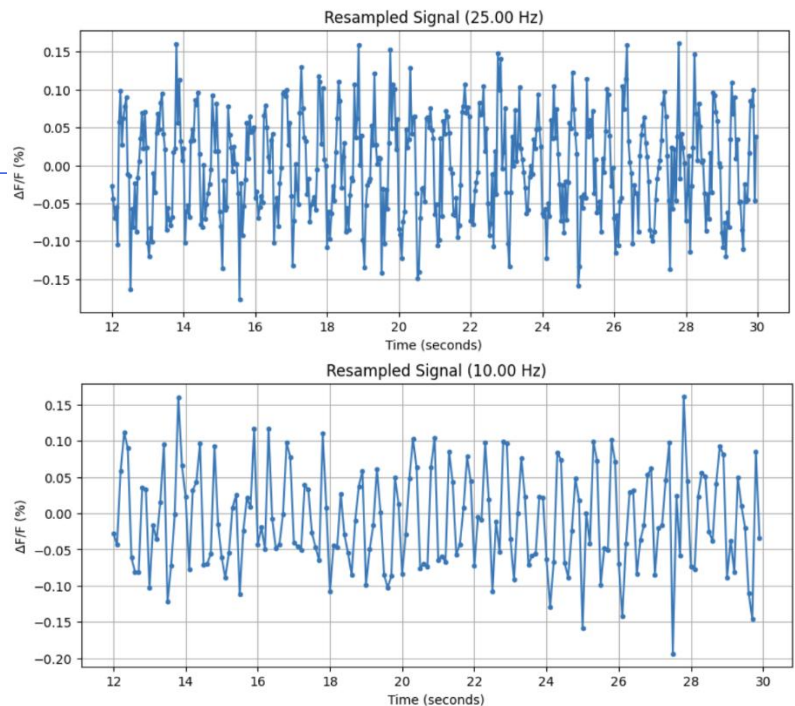
    # calculate step size (how many samples to skip)
    step = round(HZ / f_new)

    # actual sampling rate after rounding
    actual_fs = HZ / step

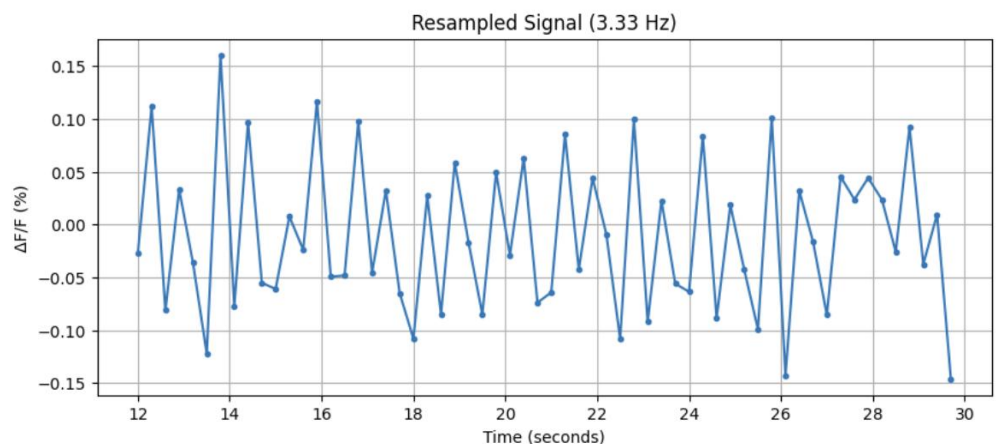
    # downsample by taking every 'step'
    resampled_data = stimulus_part[::step]

    # create new time axis
    time_resampled = np.arange(len(resampled_data)) / actual_fs + stim_start_time

    # plot the resampled signal
```



In contrast, when the sampling rate drops to ≈ 3.33 Hz, the time resolution decreases, and it becomes harder to represent fast changes in the signal:



According to the Nyquist theorem, a sampling rate of at least twice the maximum frequency in the signal is required. At the low rate of ≈ 3.33 Hz this condition is barely met, and therefore information is lost and even aliasing occurs; this phenomenon is expressed in the signal appearing thinner and less accurate, which harms the reliability of the analysis.

C.1 Effect of Sampling Rate and Aliasing on Frequency Identification

For each sampling rate we performed resampling (Downsampling) and estimated the observed frequency by counting peaks over 12 to 30 seconds of the stimulus. In addition, we compared the results to the calculation of the aliasing frequency: $f_{\text{alias}} = |f - n \cdot f_s|$.

According to the results, it can be seen that for sampling frequencies of 10 and 25 Hz, the Nyquist condition $f_s > 2 \cdot f_{\text{signal}}$ is met, so there is no aliasing, and the number of peaks in the graph matches the original frequency of the signal.

In contrast, for a sampling rate of 3.33 Hz, the Nyquist condition is not met, since $3.33 < 4$ Hz, and as a result aliasing occurs. According to the calculation we obtain: $f_{\text{alias}} = |2 - 3.33| = 1.33$ Hz.

This result matches the number of peaks actually observed in the graph (about 24 peaks over 18 seconds), i.e., the measured frequency is lower than the original frequency. Hence it can be seen that with insufficient sampling the signal changes and receives an incorrect representation of a lower frequency, even though the original frequency did not change.

Code snippet and results:

```
f_signal = 2.0
duration = 18.0

print(f"{'Actual Fs':<12} | {'Calculation (N)':<18} | {'f_alias (Hz)':<15} | {'Exp. Peaks'}")
print("-" * 65)

for f_target in target_frequencies:

    # compute downsampling step size
    step = round(HZ / f_target)

    # actual sampling frequency after rounding
    actual_fs = HZ / step

    # check Nyquist condition
    if actual_fs > 2 * f_signal:
        f_alias = f_signal
        calc_str = "No Aliasing"
    else:
        # compute possible aliasing candidates
        possible_aliases = []
        for n in [1, 2, 3]:
            val = abs(f_signal - n * actual_fs)
            possible_aliases.append(val)

        # choose smallest alias frequency
        f_alias = min(possible_aliases)

        # index of selected n
        n_used = possible_aliases.index(f_alias) + 1

        calc_str = f"|{f_signal} - {n_used}*{actual_fs:.2f}|"

    # expected number of peaks over full duration
    expected_peaks = f_alias * duration

    print(f"{'actual_fs':<12.2f} | {'calc_str':<18} | {'f_alias':<15.2f} | {'expected_peaks':<15.1f}")
```

Actual Fs	Calculation (N)	f_alias (Hz)	Exp. Peaks
25.00	No Aliasing	2.00	36.0
10.00	No Aliasing	2.00	36.0
3.33	2.0 - 1*3.33	1.33	24.0

D.1 Quantization and the Minimum Number of Bits Required

In this section, we had to calculate how many bits are required to distinguish a small change of 0.1% in the signal amplitude. First we convert the percentage to a decimal fraction: $0.1\% = 0.001$, i.e., we want the system to be able to distinguish between levels whose size is about one part in a thousand.

Then we use the fact that a system with n bits can represent 2^n different levels. Therefore, we look for the minimal value of n that satisfies $2^n \geq 1000$. In the code we calculate this using a base-2 logarithm, i.e., $\log_2(1/0.001)$, and then round up to obtain a whole number of bits. Code and result display:

```
s def calculate_min_bits(resolution_percent):  
    resolution_fraction = resolution_percent / 100  
    n_bits = np.ceil(np.log2(1 / resolution_fraction))  
    return int(n_bits)  
  
required_bits = calculate_min_bits(0.1)  
  
print("\nQuantization Analysis:")  
print(f"To distinguish a 0.1% change, we need at least {required_bits} bits.")  
  
Quantization Analysis:  
To distinguish a 0.1% change, we need at least 10 bits.
```

Thus we obtained the result that at least 10 bits are required for the system to be able to distinguish a change of 0.1% reliably.

Part 2

1. SNR Calculation Using Two Methods and Comparison of the Results

In this section we calculated the signal-to-noise ratio (SNR_{rms}) using two different methods learned in class.

In the first method, the “Power Subtraction Method”, we calculated the noise power from the baseline segment:

$$P_{noise} = \text{mean}(\text{noise}^2)$$

and the total power of the stimulus segment:

$$P_{recorded} = \text{mean}(\text{signal}^2)$$

Since the recorded signal includes both the signal itself and noise, we subtracted the noise to obtain the power of the clean signal:

$$P_{signal} = P_{recorded} - P_{noise}$$

and then we calculated:

$$SNR_{rms} = \sqrt{\frac{P_{signal}}{P_{noise}}}$$

In the second method, the “Amplitude Method”, we calculated the ratio between the mean of the signal in the stimulus segment and the standard deviation of the noise:

$$SNR = \frac{\mu_{signal}}{\sigma_{noise}}$$

Code snippet and results:

```
P_noise = np.mean(noise_signal**2)
P_recorded = np.mean(stim_signal**2)

P_signal_pure = P_recorded - P_noise
if P_signal_pure < 0: P_signal_pure = 0

snr_rms_way1 = np.sqrt(P_signal_pure / P_noise)

snr_rms_way2 = np.mean(stim_signal) / np.std(noise_signal)

print(f"Method 1 - Power Subtraction: {snr_rms_way1:.4f}")
print(f"Method 2 - Amplitude: {snr_rms_way2:.4f}")
```

```
Method 1 - Power Subtraction: 1.3376
Method 2 - Amplitude: -0.0231
```

Different results were obtained between the two methods due to the nature of the signal. The signal in the stimulus segment is an oscillatory signal around zero, and therefore in the amplitude method the positive and negative oscillations tend to cancel each other out. As a result, a very small and even negative value was obtained, which does not properly represent the true strength of the signal's activity.

In contrast, in the first method a positive value was obtained, because squaring turns all values positive and therefore measures the total energy of the signal regardless of the direction of oscillation.

2. The Effect of Block Averaging on Noise Reduction and SNR Improvement

In this section we performed noise reduction using Block Averaging, i.e., dividing the signal into segments (of length 2, 3 and 5 samples) and computing the mean of each segment instead of a moving window. In the code itself, the samples were trimmed so that the length is divisible by n , and then a reshape was performed and the mean of each block was computed. This produces a signal with a lower effective frequency, in which fast oscillations (mainly noise) are smoothed out.

Then the SNR was computed for each version of the signal using an RMS-based function, which calculates the signal and noise power through the mean of squares and separates them by subtracting the powers. The results show that the SNR increases as n increases, i.e., the stronger the smoothing.

Code display:

```
def calc_snr(signal_samples, noise_samples):
    # compute signal and noise power (RMS based)
    S2 = np.mean(signal_samples**2)
    N2 = np.mean(noise_samples**2)

    # extract pure signal power by subtracting noise
    P_signal = S2 - N2
    P_signal = max(P_signal, 0)

    # compute SNR (avoid division by zero)
    return np.sqrt(P_signal / N2) if N2 > 0 else 0

def block_average(data, n):
    # trim data so it can be divided into equal blocks
    length = len(data) // n
    trimmed = data[:length * n]

    # reshape into blocks and average each block
    return trimmed.reshape(-1, n).mean(axis=1)

# compute original SNR
original_snr = calc_snr(stim_signal, noise_signal)

factors = [2, 3, 5]

plt.figure(figsize=(12, 6))

# time axis for original signal
time_orig = (np.arange(len(stim_signal)) / HZ) + 12
plt.plot(time_orig, stim_signal, alpha=0.3, label='Original (50Hz)')

for n in factors:

    # apply block averaging to signal and noise
    signal_avg = block_average(stim_signal, n)
    noise_avg = block_average(noise_signal, n)

    # compute SNR after averaging
    snr_new = calc_snr(signal_avg, noise_avg)

    # adjust time axis according to new sampling rate
    time_avg = np.arange(len(signal_avg)) * (n / HZ) + 12

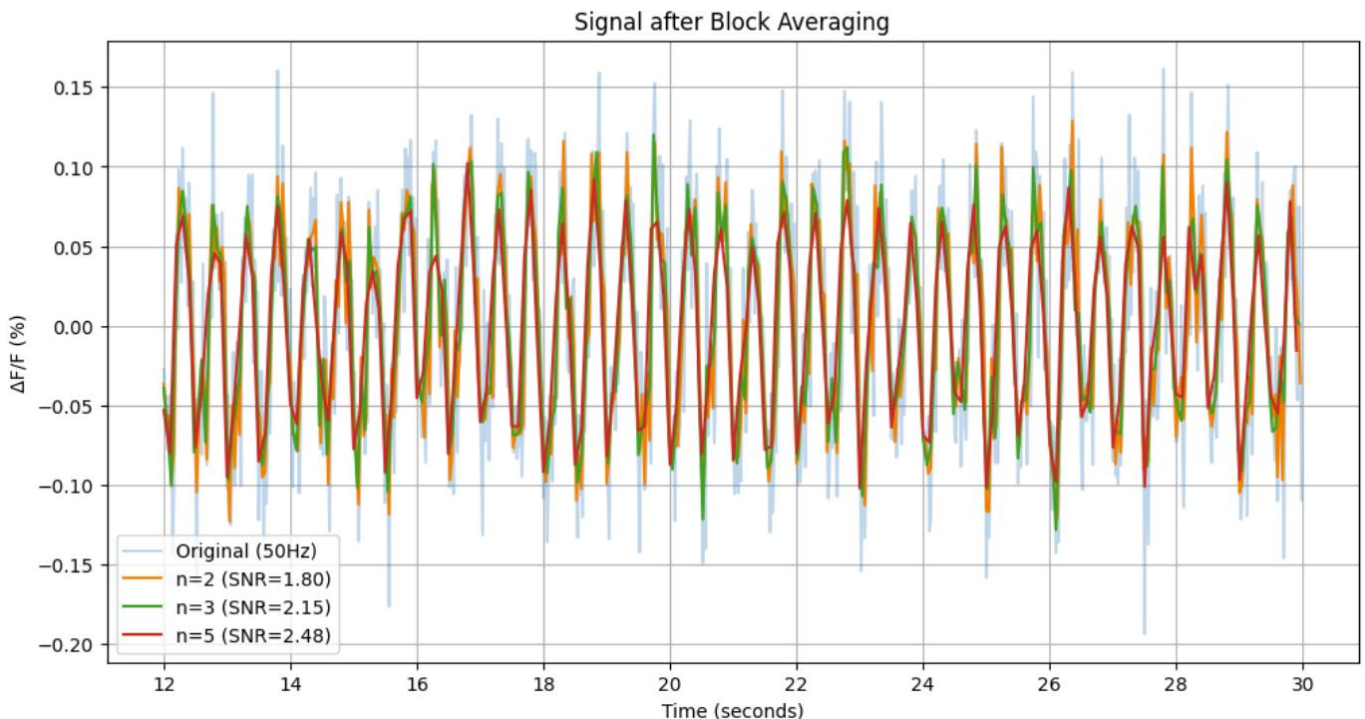
    # plot averaged signal
    plt.plot(time_avg, signal_avg, label=f'n={n} (SNR={snr_new:.2f})')

    print(f"After averaging every {n} points: SNR = {snr_new:.4f}")
```

From the graph it can be seen that the original signal (in light blue) is noisier and more oscillatory, whereas after averaging with $n=2,3,5$ the signal becomes gradually smoother. Thus, the larger n is, the stronger the smoothing obtained, which reduces noise but also causes a loss of fast details in the signal.

Graph display:

After averaging every 2 points: SNR = 1.8021
After averaging every 3 points: SNR = 2.1533
After averaging every 5 points: SNR = 2.4823



3. Noise Filtering Using a 20-Sample Average

In this section we performed noise reduction using Block Averaging, i.e., dividing the original signal into blocks of 20 samples and computing the mean of each block.

```
n = 20

# compute block averaged signal
signal_avg_20 = block_average(stim_signal, n)

# effective sampling rate after averaging
new_fs = HZ / n

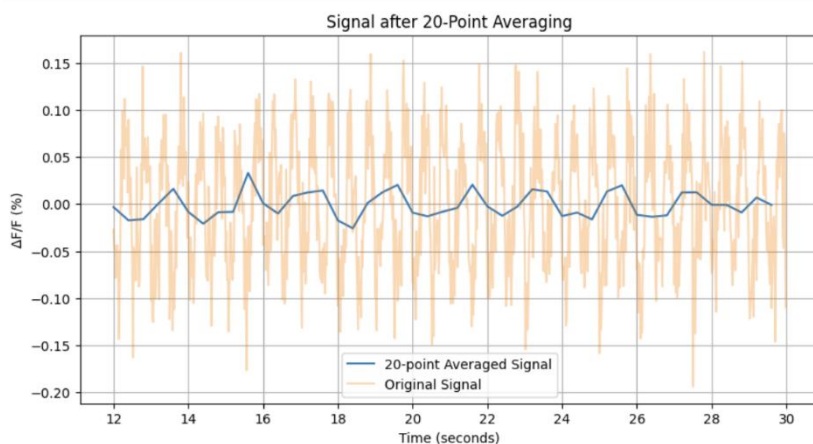
# time axis for the downsampled signal
time_avg_20 = np.arange(len(signal_avg_20)) * (n / HZ) + 12

plt.figure(figsize=(10, 5))

# plot averaged smoothed signal
plt.plot(time_avg_20, signal_avg_20, label='20-point Averaged Signal')

# original signal for comparison
time_orig = (np.arange(len(stim_signal)) / HZ) + 12
plt.plot(time_orig, stim_signal, alpha=0.3, label='Original Signal')
```

In the code, we defined $n=20$, and then the signal was trimmed so that the number of samples fits a complete division into blocks. Then, a reshape was performed into a structure of computational blocks, and computing the mean of each block created a new signal at a lower sampling rate.



In the graph it can be seen that the averaged signal (blue) is significantly smoother compared to the original signal (yellow). The reason we can no longer see the original oscillations is that averaging over 20 points acts as a low pass effect, i.e., fast oscillations are greatly attenuated and even canceled, while only slow changes are preserved. In addition, the reduction in temporal resolution causes a loss of information at high

frequencies, and therefore the original oscillations disappear visually.

4. Smoothing the Signal Using a Double Rectangular Kernel

In this section we smoothed the signal using convolution with a rectangular kernel of size 3, twice in a row. First we defined an averaging kernel in which each of the three points receives equal weight, and then convolution of the signal with the kernel was performed once more on the signal that had already been smoothed. This operation creates a low pass effect, which suppresses fast oscillations in the signal and reduces noise. Code display:

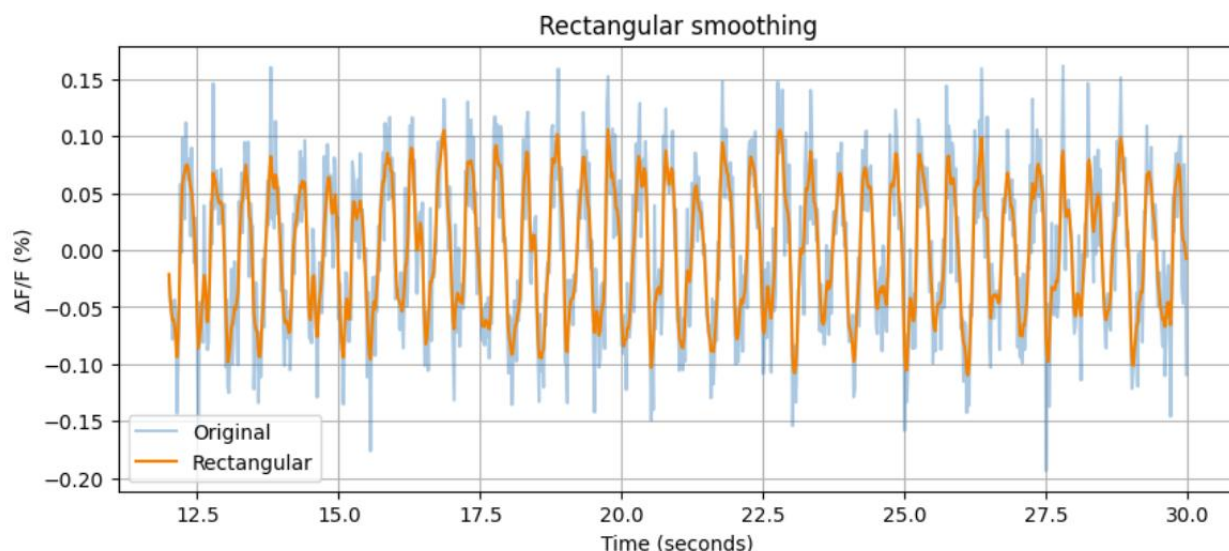
```
# rectangular smoothing kernel
rect_kernel = np.ones(3)

# normalize kernel so sum = 1
rect_kernel = rect_kernel / np.sum(rect_kernel)

# first smoothing pass (moving average)
rect_once = np.convolve(stim_signal, rect_kernel, mode='same')

# second smoothing pass
rect_twice = np.convolve(rect_once, rect_kernel, mode='same')
```

From the graph it can be seen that after two smoothing operations (orange), the signal is significantly smoother relative to the original signal (light blue). The fast oscillations almost disappear, while the general trend of the signal is preserved. However, the double smoothing also causes a loss of fast details in the signal, so there is a trade-off here between noise reduction and loss of temporal resolution.



5. Smoothing the Signal Using a Triangular Kernel

In this section we smoothed the signal using a triangular kernel of size 5. The kernel is defined so that values near the center receive a higher weight, while values at the edges receive a lower weight, and it is then normalized so that the sum of the weights is 1. After building the kernel, we performed convolution with the original signal, and thus a smoothed signal was obtained.

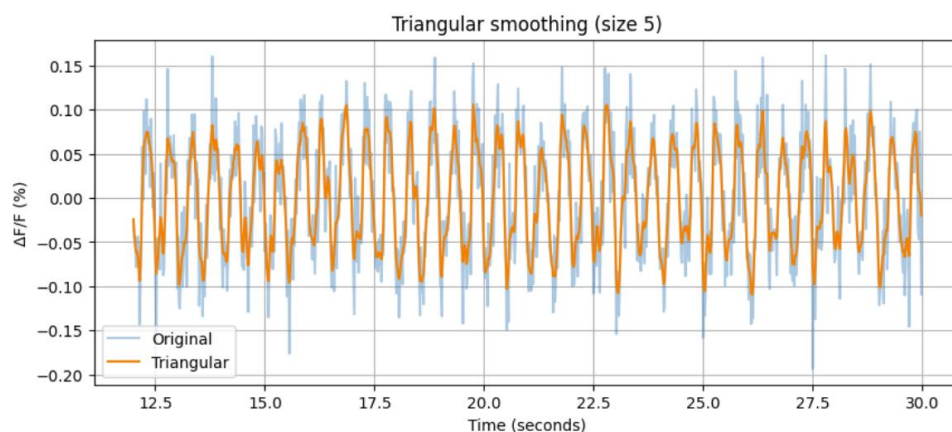
Code snippet:

```
# define triangular kernel (higher weight in center)
tri_kernel = np.array([1, 2, 3, 2, 1])

# normalize kernel so sum = 1
tri_kernel = tri_kernel / tri_kernel.sum()

# apply convolution for smoothing
tri_smoothed = np.convolve(stim_signal, tri_kernel, mode='same')
```

From the graph it can be seen that the signal after smoothing (blue) is smoother than the original signal (light blue), but still retains the general structure of the changes in the signal. Graph display:



6. Comparison of Smoothing Methods

```
plt.figure(figsize=(12,6))

plt.title("Comparison of smoothing methods")
plt.xlabel("Time (seconds)")
plt.ylabel("ΔF/F (%)")

# original noisy signal
plt.plot(time_stim, stim_signal, alpha=0.4, label="Original")

# rectangular smoothing result
plt.plot(time_stim, rect_twice, label="Rectangular")

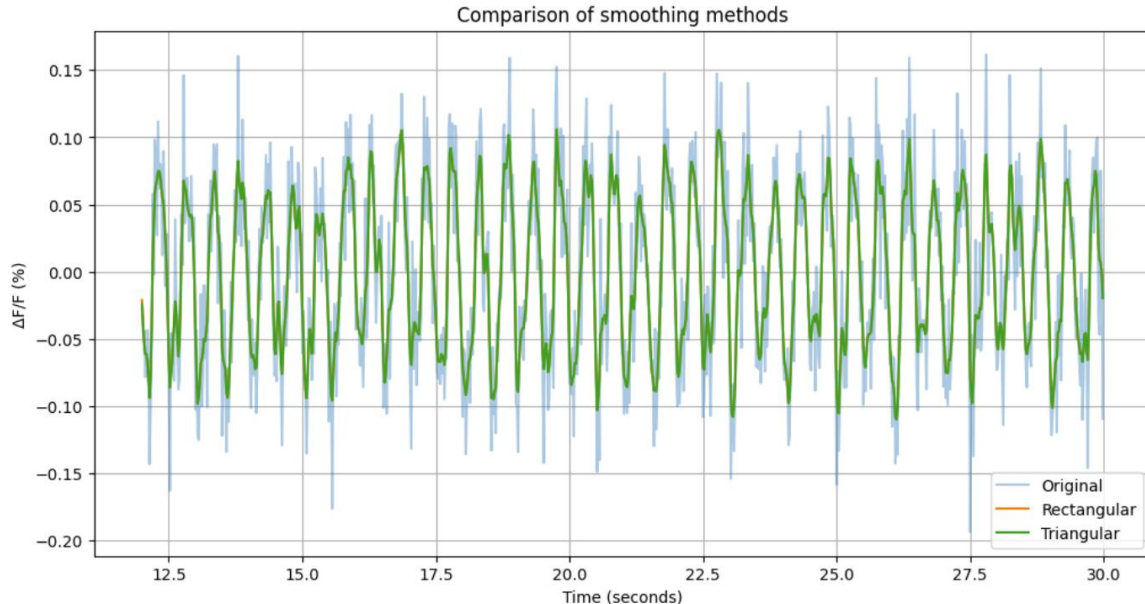
# triangular smoothing result
plt.plot(time_stim, tri_smoothed, label="Triangular")

plt.grid(True)
plt.legend()
plt.show()
```

In this section we compared three signals: the original signal, the signal after smoothing with a rectangular kernel, and the signal after smoothing with a triangular kernel. In the code, a joint display of the three signals was performed on the same graph in order to allow a direct comparison between the filtering methods. Code display:

It can be seen that the graphs of the double rectangular smoothing and of the triangular smoothing overlap completely, so that in practice only one line (green) is obtained, while the original signal remains more oscillatory. The reason for this is that, according to the associativity property of convolution, the following holds: $(x * w_1) * w_2 = x * (w_1 * w_2)$.

That is, smoothing a signal twice in a row is equivalent to a single smoothing with a new kernel obtained from the convolution between the two kernels. In our case, we performed smoothing twice with the rectangular kernel $[1, 1, 1]$. When convolving this kernel with itself, we obtain the triangular kernel $[1, 2, 3, 2, 1]$, which is exactly the kernel we used in the triangular smoothing. Therefore, both methods actually perform the same filtering operation on the signal, and so completely identical graphs are obtained.



7. Low Pass Filtering

The smoothing operation is considered a Low Pass Filter, since it suppresses fast oscillations and noise (high frequencies) and allows only low frequencies to pass, i.e., the slow changes of the signal. Mathematically, the smoothing operation computes the average between each point and the points around it. When computing an average, positive and negative values of fast oscillations tend to cancel each other out, and therefore the noise is reduced. In contrast, slow changes in the signal hardly change during averaging and are therefore preserved. Therefore, smoothing weakens components with a high frequency and preserves components with a low frequency, and a smoother signal is obtained that represents only the general trend.

Part 3

A.1 Mean Firing Rate

In this section we calculated the mean firing rate (Mean Firing Rate) of each neuron across all trials. First, the neuron type (A–E) was identified from the file. Then, for each trial the total number of spikes was counted by summing all the values of 1 in the data row, where each 1 represents a firing of the neuron. We divided the number of spikes by the recording duration (30 seconds) in order to obtain the firing rate in units of Hz, i.e., the number of spikes per second.

Finally, an average of the firing rate across all trials of each neuron was computed using groupby, and thus the mean firing rate for each neuron was obtained. The results allow comparing the activity levels of the different neurons and identifying which neurons fire at higher frequencies. Code and results display:

```
trial_duration_sec = 30

# extract neuron label (A-E)
df3['neuron'] = df3['trial_neuron'].str.extract(r'([A-E])', expand=False)

# count spikes in each trial and convert to firing rate (Hz)
df3['trial_fr'] = (df3.drop(columns=['trial_neuron', 'neuron']).sum(axis=1) / trial_duration_sec)

# compute mean firing rate across trials for each neuron
fr_results = (df3.groupby('neuron')['trial_fr'].mean().to_frame(name='Mean Firing Rate (Hz)'))

print(fr_results)
```

neuron	Mean Firing Rate (Hz)
A	11.670333
B	42.550000
C	100.000000
D	49.984333
E	28.679000

B.1 Fano Factor

In this section we calculated the Fano Factor for each neuron. The calculation was performed over a fixed time window of one second, i.e., 1000 samples at a sampling rate of 1000 Hz. In the code we selected the first 1000 columns, which represent the first second of each trial. Then the number of spikes in each trial was counted by summing the values of 1, and saved as spike_count. For each neuron, the mean and the variance of the number of spikes across trials were computed, and finally the Fano Factor was calculated according to:

$$FF = \frac{Var}{Mean}$$

Code and results display:

```
# take 1 second window (1000 samples)
spike_data = df3.iloc[:, 1:1001]

# count spikes per trial
df3['spike_count'] = spike_data.sum(axis=1)

# calculate FF per neuron
ff_results = df3.groupby('neuron')['spike_count'].agg(['mean', 'var'])

ff_results['Fano_Factor'] = ff_results['var'] / ff_results['mean']

print(ff_results['Fano_Factor'])
```

neuron	
A	0.986133
B	0.574943
C	0.000000
D	0.010218
E	6.653800

Name: Fano_Factor, dtype: float64

Analysis of the results shows that the Fano Factor can be used to characterize the firing activity type of each neuron. When neuron A obtained $FF \approx 0.99$, a value very close to 1, it fits a Poisson model in which firing is random and independent. Neuron B obtained $FF \approx 0.57$, i.e., variance lower than the mean, which indicates relatively regular firing. Neurons C and D obtained values very close to 0 ($FF=0$ and $FF \approx 0.01$), and therefore they display almost completely constant firing across the trials, with minimal variance in the number of spikes. In contrast, neuron E obtained $FF \approx 6.65$, a high value indicating Bursting firing, i.e., periods of increased activity alongside quiet periods, which creates extremely high variance between the different windows.

C.1 Fano Factor – Neuron A

As explained in the previous section, neuron A is of the Poisson model type, with an FF close to 1.

D.1 ISI distribution – Neurons A, B

In this section we calculated and displayed the distribution of ISI times for neurons A and B. In the code we first isolated all the spikes of each neuron, and then calculated the intervals between consecutive spikes using `np.diff`. Since the sampling rate is 1000 Hz, each distance between indices was converted to seconds by multiplying by 1/1000. Finally, all the time intervals from all the trials were combined, and a histogram describing the ISI distribution was built for them. Code display:

```
new_HZ = 1000

# keep only spike columns
spike_data = df3.drop(columns=['trial_neuron', 'neuron', 'trial_fr', 'spike_count'])

# get trials of neuron A and B
neuron_A_trials = spike_data[df3['neuron'] == 'A'].values
neuron_B_trials = spike_data[df3['neuron'] == 'B'].values

# function to plot ISI distribution
def ISI_plot(data, title):

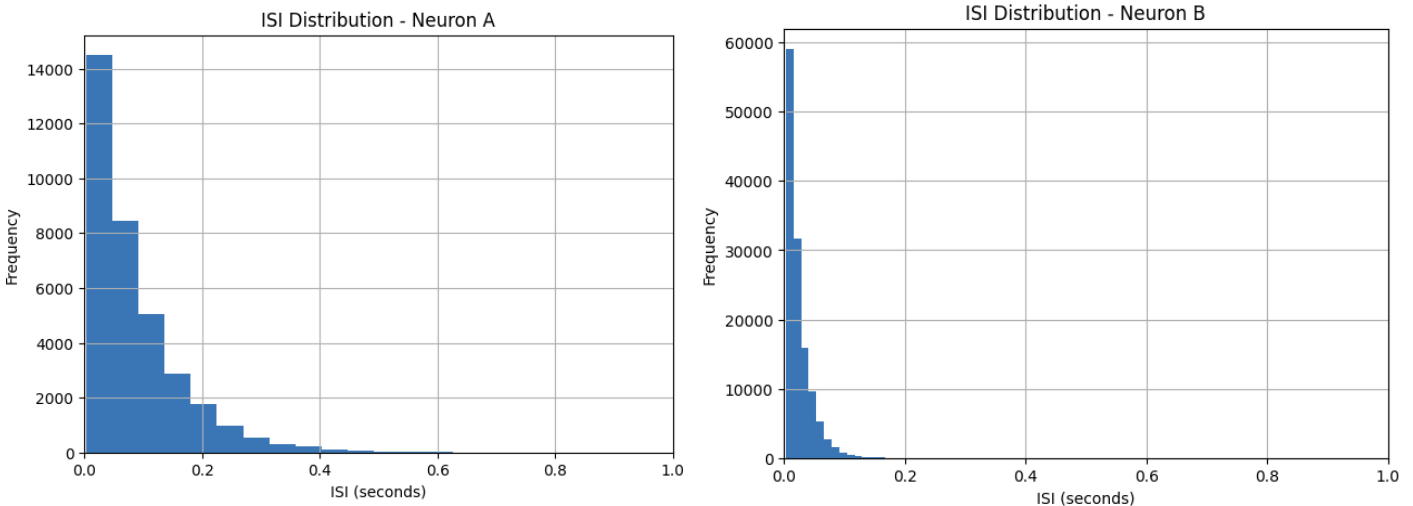
    all_distances = []

    # time per sample
    bin_size = 1 / new_HZ

    for trial in data:

        # indices where spike = 1
        spike_positions = [i for i, val in enumerate(trial) if val]
        # differences between consecutive spikes
        distances = np.diff(spike_positions)
        # convert from samples to seconds
        distances = distances * bin_size
        # add to list
        all_distances += list(distances)
```

From the results it can be seen that neuron A displays a broad, exponential ISI distribution, with many short intervals and few long intervals. This shape is typical of Poisson firing, in which the probability of a spike does not depend on the time elapsed since the previous spike, and therefore high variance is obtained in the firing times. In contrast, neuron B displays a narrow distribution that is more concentrated around a certain ISI value, which is expressed as a peak in the graph. This pattern indicates regular firing (Regular firing), in which the neuron maintains a relatively stable firing rate with low variance in the time intervals between spikes. Graph display:



E.1 Raster Plot – Neurons A, C, D, E

```
neurons_to_plot = ['A', 'C', 'D', 'E']

for neuron in neurons_to_plot:

    # get trials of current neuron
    neuron_trials = spike_data[df3['neuron'] == neuron].values

    plt.figure(figsize=(10, 5))

    # go over all trials
    for trial_index, trial in enumerate(neuron_trials):

        # find spike locations
        spike_times = np.where(trial[:new_HZ] == 1)[0]

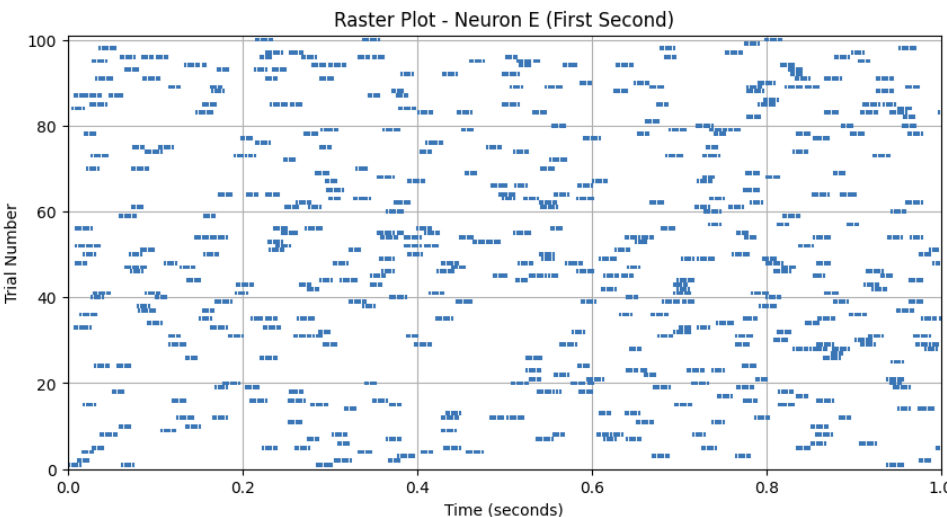
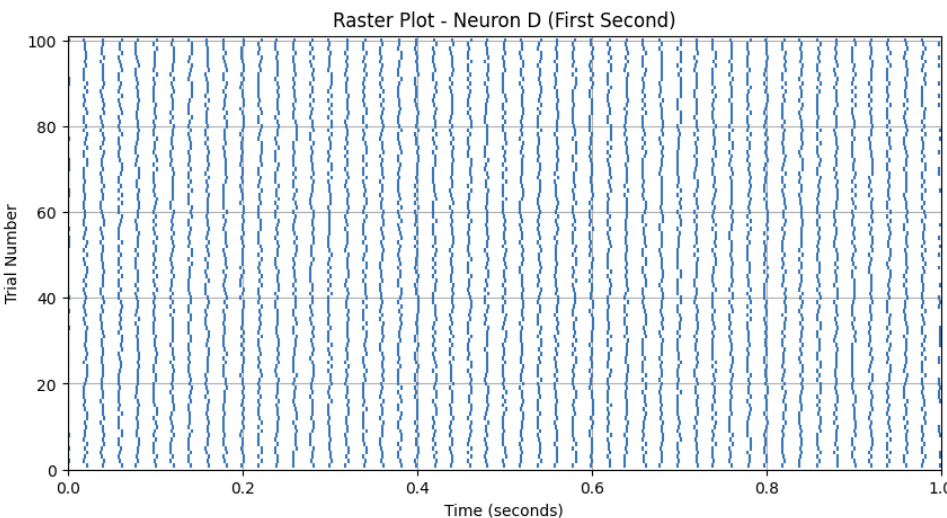
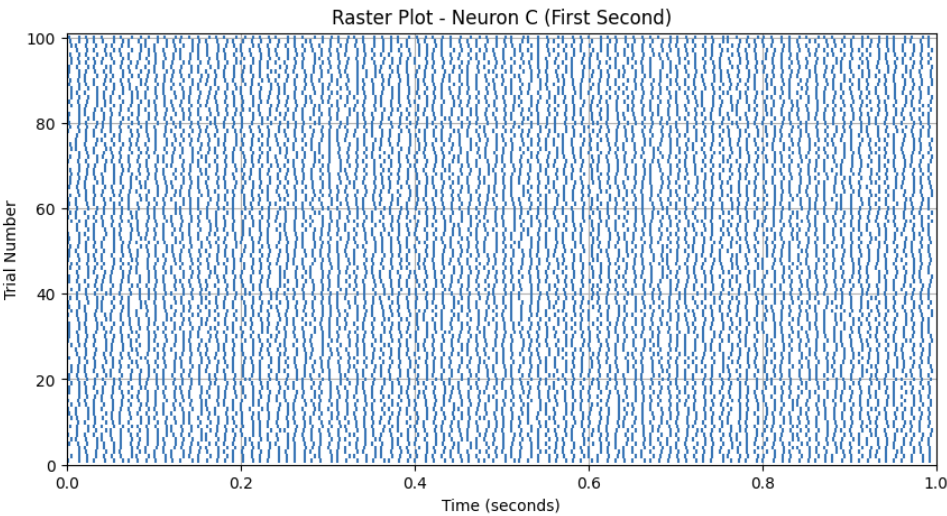
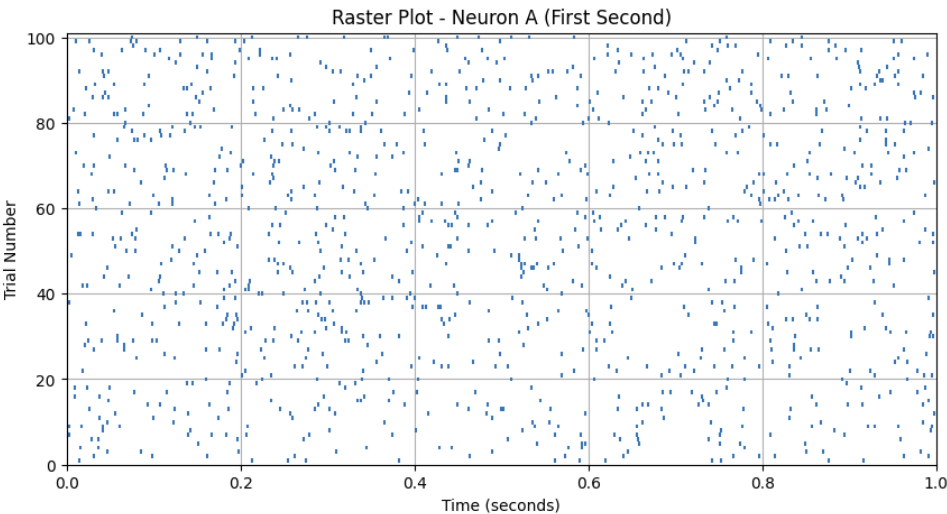
        # convert to seconds
        spike_times = spike_times / new_HZ

        # draw raster lines
        plt.vlines(spike_times, trial_index + 0.5, trial_index + 1.5)
```

In this section we created a Raster Plot for neurons A, C, D and E, focusing on the first second of the recording. In the code we went over each neuron separately, selected all of its trials, and then for each trial located the spike times using `np.where`. Since the sampling rate is 1000 Hz, the indices were converted to seconds by dividing by 1000. Finally, each spike was displayed as a vertical line (vlines) at the appropriate time position and in the row of the corresponding trial, and thus a visual representation of the firing pattern across the trials was obtained.

From the graphs, different firing patterns can be seen between the neurons. Neuron A displays relatively random firing with no fixed structure, which fits Poisson behavior. Neurons C and D display regular and consistent firing, with the spikes appearing at almost the same times in every trial, and therefore orderly vertical lines are obtained. Neuron D displays slightly more variance in the firing time. In contrast, neuron E displays a burst firing pattern – groups of dense spikes appearing at certain times alongside quiet periods.

Graph display:



F.1 Classification of Neuron E

As explained in the previous section, neuron E is a Bursting neuron (firing in bursts). This can be seen in the Raster Plot in that the spikes are not evenly distributed over time, but rather appear in dense groups of fast firing, between which there are relatively quiet periods. In addition, this result is also consistent with the Fano Factor value we obtained for neuron E ($FF \approx 6.65$), which indicates high variance in the number of spikes between different time windows.

G.1 PSTH Histogram

```
# normalization settings
neurons_psth = ['A', 'D']
num_trials = 100
num_bins = 200

# bin width in seconds (5 ms)
bin_width_sec = (1000 / num_bins) / 1000

for neuron in neurons_psth:

    # get current neuron trials
    neuron_trials = spike_data[df3['neuron'] == neuron].values

    # keep only first second
    first_second = neuron_trials[:, :new_HZ]

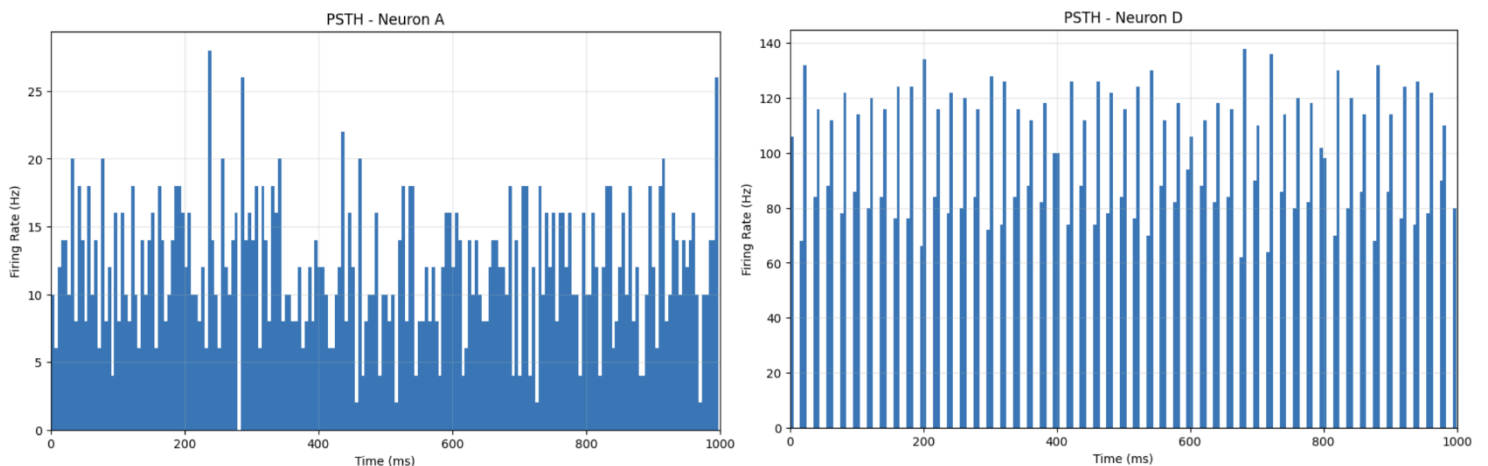
    # collect spike times from all trials
    histogram_data = []

    for trial in first_second:
        spike_positions = np.where(trial == 1)[0]
        histogram_data += list(spike_positions)

    # normalize histogram to firing rate (Hz)
    weights = np.ones_like(histogram_data) / (num_trials * bin_width_sec)
```

In this section we created a PSTH graph for neurons A and D during the first second of the recording, with the Y axis normalized to Firing Rate in units of Hz. In the code, all the trials of each neuron were selected, and from each trial the spike times were collected using `np.where`. Then all the firing times were combined into one list, and a histogram with 200 bins was built, so that each bin represents a time window of 5 ms. To convert the number of spikes to a firing rate in Hz, normalization was performed by dividing each spike by the number of trials and by the bin width in seconds (weights).

From the results it can be seen that neuron A displays relatively uniform activity over time, without sharp peaks, which fits random Poisson firing. In contrast, neuron D displays high and sharp peaks at certain times, i.e., many spikes appear in the same time window in a large number of trials. Therefore, high instantaneous firing rate values are obtained. The reason the peaks in neuron D are higher than the mean firing rate is that the PSTH measures an instantaneous firing rate within short time windows, and not an average over the entire recording. When many trials are synchronized to the same time, a high peak is obtained even if the overall mean firing rate of the neuron is lower.



A.2 Autocorrelation Plot

```
duration_2_min_ms = 2 * 60 * 1000
max_lag_ms = 100

neurons_to_analyze = ['A', 'C', 'D', 'E']

for neuron_name in neurons_to_analyze:

    # get all trials of current neuron
    neuron_trials = spike_data[df3['neuron'] == neuron_name].values

    # concatenate trials into one long signal
    flattened_signal = neuron_trials.flatten()

    # keep first 2 minutes
    signal_segment = flattened_signal[:duration_2_min_ms]

    # compute autocorrelation
    corr = np.correlate(signal_segment, signal_segment, mode='full')

    mid_point = len(corr) // 2

    # create lag axis in ms
    lags = np.arange(-max_lag_ms, max_lag_ms + 1)

    # keep only small lag window around zero
    corr_segment = corr[
        mid_point - max_lag_ms : mid_point + max_lag_ms + 1
    ].astype(float)

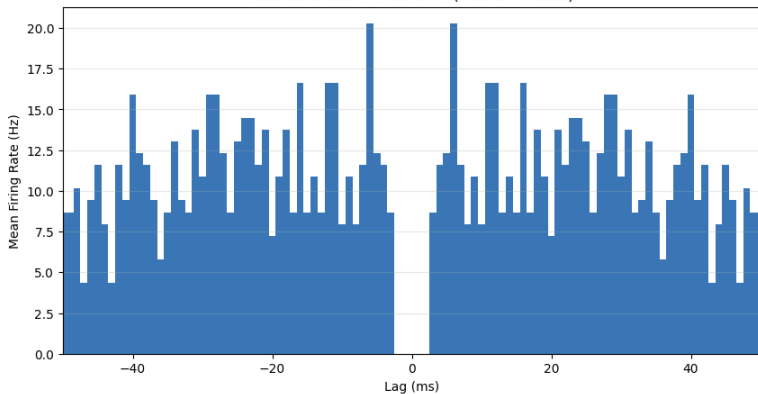
    # normalize to mean firing rate
    num_spikes = np.sum(signal_segment)
    p_firing = corr_segment / num_spikes
    fr_hz = p_firing * new_HZ

    # remove self-correlation peak at lag 0
    fr_hz[max_lag_ms] = 0
```

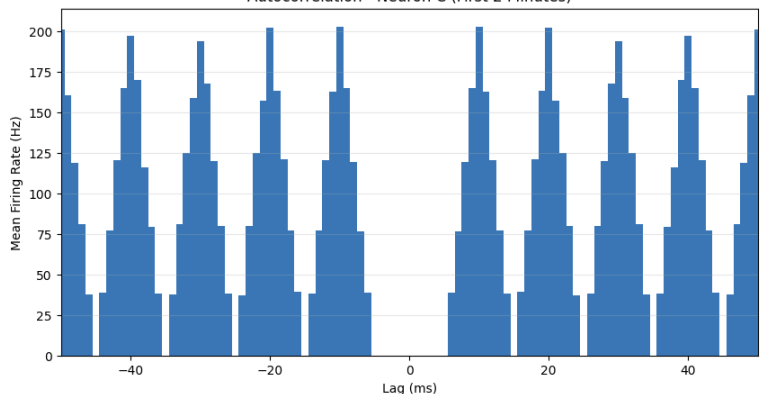
In this section we calculated and displayed the autocorrelation for neurons A, C, D and E over the first two minutes of the recording. In the code, all the trials of each neuron were combined into one long time series, and from it 120,000 samples corresponding to 2 minutes at a sampling rate of 1000 Hz were taken. Then we calculated the autocorrelation using `np.correlate`. In order to normalize the result to the Mean Firing Rate, we divided the number of matches by the total number of spikes and then multiplied by the sampling rate. In addition, the value at lag=0 was zeroed.

From the graphs, different firing patterns can be seen between the neurons. Neuron A displays a relatively flat autocorrelation around zero. In the graph a relatively flat line appears immediately after the refractory period, which is typical of Poisson firing without periodicity. Neuron C displays periodic and regular peaks at fixed intervals. Neuron D also displays a periodic structure, but with slightly higher variance, and therefore the peaks are more widely spaced. In contrast, neuron E displays a strong peak around lag=0, which indicates Burst firing. In addition, very close to lag=0 a sharp drop in activity can be seen, representing the refractory period.

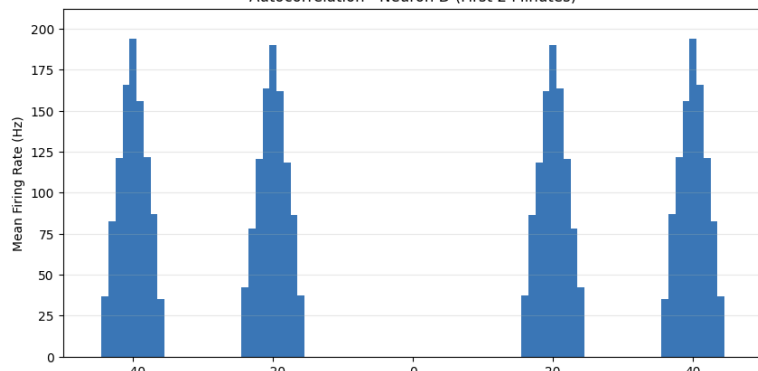
Autocorrelation - Neuron A (First 2 Minutes)



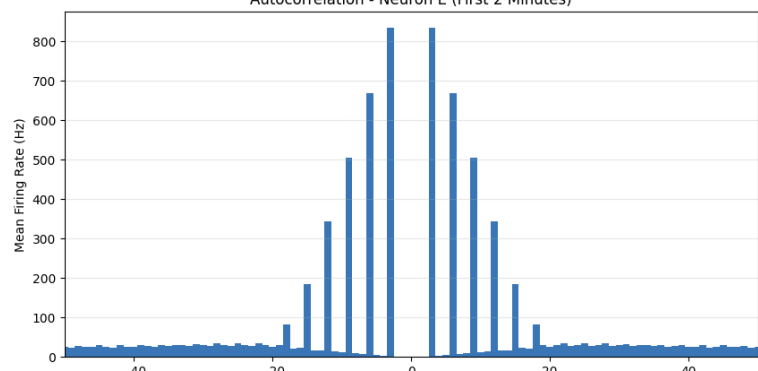
Autocorrelation - Neuron C (First 2 Minutes)



Autocorrelation - Neuron D (First 2 Minutes)



Autocorrelation - Neuron E (First 2 Minutes)



B.2 Autocorrelation – Neuron B Classification

As we saw in the previous section, neuron C is a regular neuron. This can be seen in the autocorrelation graph, in which sharp periodic peaks appear at fixed time intervals around lag=0. These peaks indicate that the neuron tends to fire at similar time intervals in a periodic manner. In addition, around lag=0 a sharp drop can be seen, corresponding to the refractory period, in which the neuron cannot fire immediately after a previous spike.

The Fano Factor value we obtained for neuron C was $FF=0$, and it indeed matches the hypothesis that this is a regular neuron. This value indicates that there is almost no variance in the number of spikes between the different trials, i.e., the neuron fires in an almost completely constant manner. Therefore, there is a full match between the periodic autocorrelation pattern and the FF value.

C.2 Autocorrelation – Neurons C, D Classification

Neurons C and D are regular neurons. The main difference between the autocorrelation functions of C and D is the level of regularity of the firing, where neuron D is less regular than neuron C. In neuron C, sharp, high peaks at fixed intervals are obtained. In contrast, in neuron D the peaks are less sharp and more widely spaced, i.e., it fires at a lower rate. This difference also matches the Fano Factor values: neuron C received $FF=0$, while neuron D received a slightly higher value, $FF \approx 0.01$, which indicates a small variance.

D.2 Autocorrelation – Neuron E

It can be seen that around lag=0 a wide and symmetric structure is obtained, with high peaks close to zero. This shape indicates that neuron E is a bursting neuron, i.e., a neuron that fires a sequence of dense spikes in a short time. After one spike occurs, there is a very high probability of additional spikes appearing, and therefore particularly high correlation values are obtained at small lag values around zero.

The wide triangular shape around zero reflects the duration of the burst: as one moves away from lag=0, the probability of finding another spike gradually decreases, and therefore the height of the bars also decreases. Outside this region the correlation returns almost to the baseline value of the mean firing rate.

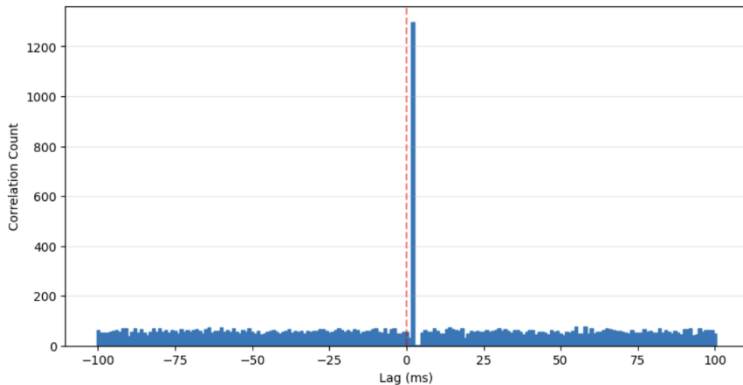
1. The number of peaks in the autocorrelation graph can indicate the firing pattern and periodicity of the neuron. The distance between the peaks represents the characteristic time interval between spikes (ISI) and therefore is also related to the firing frequency. When periodic peaks appear at fixed intervals, this indicates regular firing. In contrast, if mainly a central peak is obtained with no additional periodic peaks, this indicates random firing, as in a Poisson process, in which there is no clear periodicity. In bursting neurons, several very close peaks can be seen around (lag=0), indicating the appearance of several dense spikes in a short time within each burst.
2. The local minimum that appears after each peak in the autocorrelation indicates that after a spike is fired there is a temporary decrease in the probability of an additional firing. This phenomenon is mainly related to the refractory period. In regular neurons, the minimum after each peak also indicates the periodicity of the firing: after the neuron fires at a certain time, there is a short “quiet window” before the next spike in the cycle. Therefore a pattern of repeating peaks is obtained. The deeper and clearer the minimum, the more it can be concluded that the firing is more precise and regular, with relatively fixed time intervals between the spikes.

E.2 Cross – Correlation

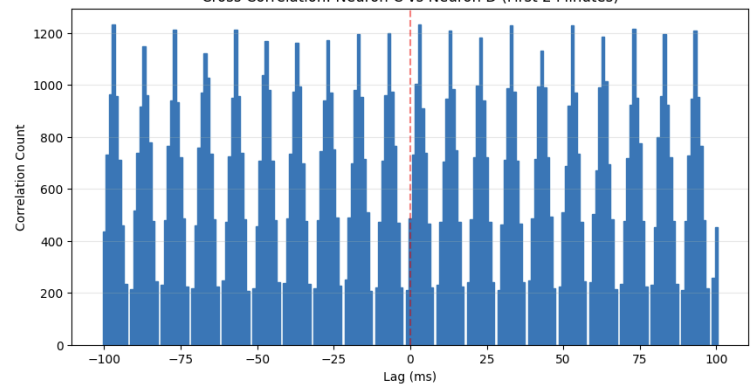
In this section we performed cross-correlation analysis for two pairs of neurons: (A, B) and (C, D). For this purpose, we used the function `get_long_recording`, which flattens (Flatten) all the trials into one long sequence and samples the first two minutes of the recording. The correlation was computed using the function `np.correlate` in mode='full', which calculates the number of overlaps (Spike counts) between the two signals for every possible shift (Lag). In accordance with the requirements, we displayed the results without normalization, while performing a “zoom” to a range of about 100 ms around the zero point in order to identify synaptic interactions.

```
def get_long_recording(neuron_label, data_df, limit_samples):  
  
    # extract trials of selected neuron  
    trials = data_df[data_df['neuron'] == neuron_label].drop(  
        columns=['trial_neuron', 'neuron', 'trial_fr', 'spike_count'],  
        errors='ignore').values  
  
    # create one long spike train  
    flattened = trials.flatten()  
  
    return flattened[:limit_samples]  
  
pairs = [('A', 'B'), ('C', 'D')]  
  
for n1, n2 in pairs:  
  
    # get 2-minute recordings  
    sig1 = get_long_recording(n1, df3, duration_2_min_ms)  
    sig2 = get_long_recording(n2, df3, duration_2_min_ms)  
  
    # compute cross-correlation  
    cc_full = np.correlate(sig2, sig1, mode='full')  
  
    lags = np.arange(-(len(sig1) - 1), len(sig1))  
    center_idx = len(cc_full) // 2  
  
    # zoom around lag=0  
    cc_zoom = cc_full[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]  
  
    lags_zoom = lags[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]
```

Cross Correlation: Neuron A vs Neuron B



Cross Correlation: Neuron C vs Neuron D (First 2 Minutes)



F.2 Cross – Correlation

1. In the pair of neurons A and B, a peak above the background level can be seen in the cross-correlation graph, which indicates a temporal relationship between the neurons and not independent random firing. Since this is a positive peak and not a dip, the relationship is excitatory, i.e., firing of neuron A increases the probability that neuron B fires immediately after it. The location of the peak at a positive Lag of about 3 ms indicates that neuron A fires first and is therefore the activating neuron, while neuron B responds after a short synaptic delay. In addition, the background level in the graph is relatively flat because neuron A displays random Poisson firing without fixed periodicity.

Between neurons C and D, a series of periodic peaks at fixed intervals of about 3 ms is obtained, with no single dominant peak around a particular lag. This pattern is not typical of a direct neuronal relationship, in which one expects to see a clear peak indicating a consistent temporal dependence between the firing of the neurons. Instead, the fixed periodicity most likely indicates a similar firing rate of the two neurons, or the influence of a shared external input. Therefore, no significant evidence was found for a direct synaptic or functional relationship between the neurons based on the results.

2. The peak amplitude in the cross-correlation graph (which is not normalized) represents the number of joint occurrences in which neuron B fired within a defined time range after the firing of neuron A. The height of the peak indicates the strength of the functional relationship between the cells and the degree of reliability with which the pre-synaptic neuron manages to activate the post-synaptic neuron. Under the assumption that every spike of neuron A causes neuron B to fire, the peak amplitude in effect reflects the total number of spikes fired by neuron A during the entire recording period. In order to estimate the firing rate of neuron A in units of Hz, we use the maximum value of the peak and divide it by the duration of the recording examined (in this case 120 seconds). This method allows us to draw information about the activity rate of one neuron from observing its correlation with another neuron, provided that the synaptic connection between them is strong and known.
3. If neuron A were to activate neuron B with a probability of only 50%, the height of the peak in the cross-correlation would be half as large. The reason is that not every spike of neuron A would lead to a matching spike in neuron B, but only half of them on average. Therefore the number of matches around the delay time would be smaller, and the peak would be lower but still appear at the same lag. That is, the location of the peak does not change, only its amplitude decreases, because the relationship between the neurons becomes less reliable.

G.2 Cross – Correlation – with Normalization

In this exercise we calculated and displayed the Cross-Correlation between neurons C and D for the first two minutes of the recording, in order to test the temporal dependence between them. The cross-correlation was calculated using `np.correlate`, and then normalization was performed so that the result represents a conditional firing rate in units of Hz, i.e., the relative probability of neuron D firing given the firing of neuron C. In the graph, the x axis represents lag and the y axis represents the strength of the relationship after normalization. From the graph it can be seen that there is no sharp and clear peak around lag=0, and therefore no real-time synchronization is obtained between C and D. In addition, the function is relatively symmetric around zero, which suggests that no neuron consistently precedes the other. However, there is relatively high activity across the entire range of lags with a periodic pattern, which may indicate general joint activity or the influence of an external stimulus, but not a direct temporal relationship between the two neurons.

```

n1, n2 = 'C', 'D'

# extract first 2 minutes of spike trains
sig1 = get_long_recording(n1, df3, duration_2_min_ms)
sig2 = get_long_recording(n2, df3, duration_2_min_ms)

# raw cross-correlation (unnormalized)
cc_raw = np.correlate(sig1, sig2, mode='full')

# number of spikes in reference neuron (normalization factor)
n_spikes_source = np.sum(sig1)

# convert correlation to conditional firing rate (Hz)
cc_normalized = (cc_raw / n_spikes_source) / bin_width_sec

# create lag axis
lags = np.arange(-(len(sig1) - 1), len(sig1))
center_idx = len(cc_raw) // 2

# zoom around zero lag
cc_zoom = cc_normalized[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]
lags_zoom = lags[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]

```

